1. University Scholarship Allocation

Purpose: Evaluate applicants against academic, attendance, and income rules; rank only eligible applicants and resolve ties consistently.

Inputs: Applicant ID/name; aggregate percentage; subject marks; attendance percentage; annual family income; scholarship rules and tie-break order.

Algorithm

1. Read the eligibility thresholds and tie-break priorities.
2. For each applicant, validate that marks and attendance are within 0–100 and income is non-negative.
3. Mark eligible only if aggregate, every required subject minimum, attendance, and income criteria all pass.
4. Sort eligible applicants by aggregate descending, then subject score total descending, attendance descending, and income ascending (example tie-break policy).
5. Assign ranks; if all tie-break fields match, assign the same rank or use applicant ID as the final deterministic rule.
6. Display eligibility, reason for rejection, rank, and award decision.

Flowchart

(START)	▱ Read rules and applicant details	Validate values
◇ All values valid? ◇	No: mark INVALID	Yes: check all eligibility rules
◇ Pass every rule? ◇	No: mark NOT ELIGIBLE	Yes: add to eligible list
Sort and assign ranks	▱ Display results	(END)

Pseudocode

```
START
INPUT rules: minAggregate, subjectMinimums, minAttendance, maxIncome
INPUT applicantList
eligibleList = empty list

FOR each applicant IN applicantList
    IF any mark < 0 OR any mark > 100 OR attendance outside 0..100
       OR income < 0 THEN
        applicant.status = "INVALID"
    ELSE IF aggregate < minAggregate
       OR any required subject mark < its minimum
       OR attendance < minAttendance
       OR income > maxIncome THEN
        applicant.status = "NOT ELIGIBLE"
    ELSE
        applicant.status = "ELIGIBLE"
        ADD applicant TO eligibleList
    END IF
END FOR

SORT eligibleList BY aggregate DESC,
    subjectScoreTotal DESC, attendance DESC,
    income ASC, applicantID ASC
ASSIGN rank using the sorted order
OUTPUT all applicants with status, reason, rank
STOP
```

Sample Output

ID	Aggregate	Attendance	Income	Result / Rank
A101	91%	96%	₹1,20,000	Eligible / 1
A102	88%	94%	₹1,00,000	Eligible / 2
A103	92%	72%	₹90,000	Not eligible: attendance

Assumptions: Example policy: aggregate ≥ 80%, each required subject ≥ 60%, attendance ≥ 75%, family income ≤ ₹2,50,000. These are illustrative values, not official rules.

2. Progressive Electricity Billing

Purpose: Calculate monthly bills from consumption slabs and fees, compare months, and flag unusual increases.

Inputs: Monthly kWh, slab rates, fixed charge, surcharge percentage, rebate rules, previous-month bills, unusual-increase threshold.

Algorithm

1. Read consumption and billing parameters for each month.
2. Validate consumption is non-negative and slab limits/rates are valid.
3. Calculate energy charge incrementally within each slab.
4. Add fixed charge; calculate surcharge according to the supplied policy; subtract eligible rebate without allowing a negative bill.
5. Compare the bill with the previous month. Flag when the increase percentage exceeds the configured threshold.

Flowchart

(START)	▱ Read slabs, fees, rebates and months	Validate consumption
◇ Valid? ◇	▱ No: show input error	Yes: compute slab energy charge
Add fixed charge and surcharge; subtract rebate	Compare with previous bill	◇ Increase above threshold? ◇
Yes: flag unusual increase	No: mark normal	▱ Display monthly bill and trend
(END)		

Pseudocode

```
START
INPUT slabLimits, slabRates, fixedCharge, surchargeRate
INPUT rebateRule, monthlyConsumption, alertPercent
FOR each month
    IF consumption < 0 THEN OUTPUT "Invalid consumption"; CONTINUE
    energyCharge = 0
    remaining = consumption
    FOR each slab in ascending order
        unitsInSlab = units falling within this slab
        energyCharge = energyCharge + unitsInSlab * slab.rate
    END FOR
    surcharge = (energyCharge + fixedCharge) * surchargeRate
    rebate = CALCULATE_REBATE(energyCharge, rebateRule)
    bill = MAX(0, energyCharge + fixedCharge + surcharge - rebate)
    IF previousBill exists THEN
        increasePercent = (bill - previousBill) / previousBill * 100
        IF increasePercent > alertPercent THEN flag = "UNUSUAL INCREASE"
        ELSE flag = "Normal"
    END IF
    OUTPUT month, energyCharge, surcharge, rebate, bill, flag
    previousBill = bill
END FOR
STOP
```

Sample Output

Month	Units	Energy	Other net charges	Bill	Status
Jan	100	₹450	₹70	₹520	Baseline
Feb	120	₹570	₹70	₹640	Normal
Mar	220	₹1,170	₹70	₹1,240	Unusual: +93.8%

Assumptions: Illustrative slab rates: first 50 units ₹2/unit, next 100 ₹4/unit, above 150 ₹6/unit; fixed charge ₹50; surcharge ₹20; no rebate. Flag increase greater than 30%.

3. Hospital Billing and Patient Classification

Purpose: Total consultation, diagnostics, and treatment charges; apply policy-based concessions and classify the account.

Inputs: Patient category; consultation fee; diagnostic items; treatment charges; concession eligibility; payment made; classification policy.

Algorithm

1. Read itemized charges and patient/payment details.
2. Reject negative charges, invalid quantities, or payment greater than the amount due unless the policy allows credit.
3. Sum consultation, diagnostic, and treatment charges.
4. Apply eligible concession according to the supplied policy and cap it at the permitted amount.
5. Calculate net bill and balance; classify using the supplied payment thresholds.

Flowchart

(START)	▱ Read charges, concession and payment	◇ Charges and payment valid? ◇
▱ No: show validation error	Yes: total all service charges	Calculate eligible concession
Compute net bill and balance	Classify payment status	▱ Print itemized bill
(END)		

Pseudocode

```
START
INPUT consultation, diagnosticItems, treatmentItems
INPUT concessionRules, amountPaid, paymentPolicy
IF any charge < 0 OR amountPaid < 0 THEN
    OUTPUT "Invalid billing data"; STOP
END IF
gross = consultation + SUM(diagnosticItems) + SUM(treatmentItems)
concession = APPLY_ELIGIBLE_CONCESSION(gross, concessionRules)
concession = MIN(concession, gross)
netBill = gross - concession
balance = netBill - amountPaid
IF balance < 0 AND overpayment not allowed THEN
    OUTPUT "Payment exceeds bill"; STOP
END IF
IF balance <= 0 THEN class = "PAID"
ELSE IF amountPaid > 0 THEN class = "PARTIALLY PAID"
ELSE class = "UNPAID"
OUTPUT gross, concession, netBill, amountPaid, balance, class
STOP
```

Sample Output

Item	Amount
Consultation	₹500
Diagnostics	₹1,200
Treatment	₹3,000
Gross total	₹4,700
Concession (10%)	−₹470
Net bill	₹4,230
Paid / balance	₹3,000 / ₹1,230
Classification	PARTIALLY PAID

Assumptions: Illustration uses a 10% concession and payment-status categories. Actual concession and patient classifications must follow the supplied hospital policy.

4. Employee Payroll and Deduction Analysis

Purpose: Calculate earnings, progressive income deductions, other deductions, and a clear salary statement.

Inputs: Basic pay, allowances, overtime hours/rate, incentives, deduction brackets, provident fund or other deductions.

Algorithm

1. Validate salary and hours as non-negative.
2. Calculate overtime pay and gross salary.
3. Calculate progressive deduction by applying each rate only to the income portion in its bracket.
4. Add other deductions and calculate net pay.
5. Print a formatted statement with each earning and deduction line.

Flowchart

(START)	▱ Read pay and deduction rules	Validate all values
◇ Valid? ◇	▱ No: show error	Yes: calculate overtime and gross pay
Apply progressive and other deductions	◇ Net pay non-negative? ◇	▱ No: flag for review
▱ Yes: print salary statement	(END)	

Pseudocode

```
START
INPUT basicPay, allowances, overtimeHours, overtimeRate, incentives
INPUT deductionBrackets, otherDeductions
IF any input < 0 THEN OUTPUT "Invalid input"; STOP
overtimePay = overtimeHours * overtimeRate
gross = basicPay + allowances + overtimePay + incentives
progressiveDeduction = 0
FOR each bracket
    taxableInBracket = portion of gross in bracket
    progressiveDeduction += taxableInBracket * bracket.rate
END FOR
totalDeductions = progressiveDeduction + SUM(otherDeductions)
netPay = gross - totalDeductions
IF netPay < 0 THEN OUTPUT "Review deduction policy"; STOP
OUTPUT formatted salary statement
STOP
```

Sample Output

Salary component	Amount
Basic pay	₹30,000
Allowances	₹6,000
Overtime	₹2,000
Incentive	₹1,500
Gross salary	₹39,500
Progressive deduction (example)	₹2,950
Other deductions	₹2,000
Net salary	₹34,550

Assumptions: Illustrative progressive deduction: 5% on first ₹20,000, 10% on next ₹15,000, and 20% above ₹35,000. This is a demonstration, not tax advice or an actual payroll tax table.

5. Multi-Plan Telecommunications Cost Optimization

Purpose: Compare mobile plans against usage, validity, and budget constraints, including excess-use charges.

Inputs: Expected calls/data/SMS, plan allowances, validity, base price, excess rates, required validity, maximum budget.

Algorithm

1. Read usage estimates and each plan's features and charges.
2. Discard plans that fail minimum validity or required coverage/allowance rules.
3. Calculate excess charges for usage beyond each allowance.
4. Calculate total cost and reject plans over budget.
5. Choose the lowest-cost eligible plan; report if no plan qualifies.

Flowchart

(START)	▱ Read usage, plans and budget	Check validity and requirements
◇ Plan meets requirements? ◇	No: skip plan	Yes: calculate excess and total cost
◇ Within budget? ◇	No: skip plan	Yes: add to eligible plans
◇ Any eligible plan? ◇	▱ No: report none	Yes: select minimum-cost plan
▱ Display comparison and recommendation	(END)	

Pseudocode

```
START
INPUT expectedCalls, expectedData, expectedSMS
INPUT requiredValidity, budget, plans
eligiblePlans = empty list
FOR each plan
    IF plan.validity < requiredValidity THEN CONTINUE
    excessCall = MAX(0, expectedCalls - plan.callAllowance) * plan.callRate
    excessData = MAX(0, expectedData - plan.dataAllowance) * plan.dataRate
    excessSMS = MAX(0, expectedSMS - plan.smsAllowance) * plan.smsRate
    totalCost = plan.basePrice + excessCall + excessData + excessSMS
    IF totalCost <= budget THEN ADD (plan, totalCost) TO eligiblePlans
END FOR
IF eligiblePlans is empty THEN OUTPUT "No eligible plan"
ELSE choose plan with minimum totalCost
OUTPUT plan, excess charges, totalCost
STOP
```

Sample Output

Plan	Base	Excess	Total	Decision
A	₹299	₹0	₹299	Eligible
B	₹199	₹80	₹279	Eligible — lowest
C	₹249	₹120	₹369	Over ₹350 budget

Assumptions: Illustrative usage and prices only. Compare like-for-like validity and included benefits before choosing a real plan.

6. Railway Fare and Reservation Calculator

Purpose: Calculate passenger fares by distance, class, concessions, fees, and cancellation policy.

Inputs: Distance, class rates, passenger ages, concession eligibility, reservation fee, taxes/fees, cancellation rules.

Algorithm

1. Read journey and passenger details.
2. Validate distance, passenger ages, class, and fee inputs.
3. Calculate base fare using distance slabs and class multiplier/rate.
4. Apply age-based concession only when eligible under the supplied policy.
5. Add reservation fees and applicable charges; calculate cancellation penalty using the stated rule.

Flowchart

(START)	▱ Read distance, class and passengers	Validate journey and ages
◇ Valid? ◇	▱ No: report error	Yes: calculate distance-slab fare
Apply eligible age concession	Add reservation fees and total passengers	◇ Cancellation requested? ◇
Yes: calculate penalty and refund	No: skip cancellation calculation	▱ Print passenger-wise fares and total
(END)		

Pseudocode

```
START
INPUT distance, travelClass, passengers, fareSlabs, reservationFee
INPUT concessionRules, cancellationPolicy
IF distance <= 0 OR invalid class OR invalid age THEN
    OUTPUT "Invalid journey/passenger data"; STOP
END IF
basePerPassenger = CALCULATE_SLAB_FARE(distance, travelClass, fareSlabs)
total = 0
FOR each passenger
    concession = GET_ELIGIBLE_CONCESSION(passenger.age, basePerPassenger)
    fare = basePerPassenger - concession + reservationFee
    total = total + fare
    OUTPUT passenger fare
END FOR
IF cancellation requested THEN
    penalty = CALCULATE_PENALTY(total, cancellationPolicy)
    refund = MAX(0, total - penalty)
    OUTPUT penalty, refund
END IF
OUTPUT total
STOP
```

Sample Output

Passenger	Base fare	Concession	Fee	Final fare
Adult	₹600	₹0	₹20	₹620
Eligible senior (example)	₹600	₹120	₹20	₹500
Total				₹1,120

Assumptions: Illustration assumes a 20% concession for one eligible passenger and a ₹20 reservation fee per passenger. Actual railway fares and concession rules must be supplied by the assignment.

7. Retail Billing with Multiple Discount Rules

Purpose: Generate an itemized invoice using quantities, category-specific taxes, threshold discounts, and a maximum concession cap.

Inputs: Product, category, unit price, quantity, category tax rate, discount thresholds/rates, maximum discount.

Algorithm

1. Read items and discount/tax rules; validate prices, quantities, and rates.
2. Calculate line subtotal and subtotal before discounts.
3. Calculate eligible threshold/category discounts and apply the maximum concession limit.
4. Calculate category-specific taxes using the assignment's specified tax base.
5. Add line totals and print itemized invoice, discount, tax, and grand total.

Flowchart

(START)	▱ Read items, tax and discount rules	Validate prices and quantities
◇ Valid? ◇	▱ No: report item error	Yes: calculate subtotal
Calculate discount and apply cap	Calculate category-specific tax	Sum grand total
▱ Print itemized invoice	(END)	

Pseudocode

```
START
INPUT items, discountRules, maxDiscount, categoryTaxRates
subtotal = 0
FOR each item
    IF unitPrice < 0 OR quantity <= 0 THEN
        OUTPUT "Invalid item"; STOP
    END IF
    lineSubtotal = unitPrice * quantity
    subtotal += lineSubtotal
END FOR
discount = CALCULATE_DISCOUNT(subtotal, discountRules)
discount = MIN(discount, maxDiscount, subtotal)
discountedSubtotal = subtotal - discount
tax = CALCULATE_CATEGORY_TAX(items, discountedSubtotal, categoryTaxRates)
grandTotal = discountedSubtotal + tax
OUTPUT itemized lines, subtotal, discount, tax, grandTotal
STOP
```

Sample Output

Item	Qty	Unit price	Line total
Notebook	2	₹50	₹100
Bag	1	₹800	₹800
Subtotal			₹900
Discount (10%)			−₹90
Tax (illustrative 5%)			₹40.50
Grand total			₹850.50

Assumptions: Example applies a 10% threshold discount and a 5% tax to the discounted subtotal. Real category tax rates and discount caps must be supplied.

8. Manufacturing Cost and Profitability Analysis

Purpose: Calculate full production cost including materials, labour, wastage, machine cost, and profit target.

Inputs: Units produced, material quantities and rates, wastage percentage, labour, machine hours/rate, selling price per unit, target profit margin.

Algorithm

1. Validate quantities, rates, and percentages.
2. Calculate base material cost and wastage allowance.
3. Calculate labour and machine costs, then total production cost and unit cost.
4. Calculate revenue and profit; compute profit margin against revenue.
5. Compare the margin/profit with the target and report whether the target is met.

Flowchart

(START)	▱ Read production and selling inputs	Validate inputs
◇ Valid? ◇	▱ No: show error	Yes: calculate materials and wastage
Add labour and machine costs	Calculate unit cost, revenue and profit	◇ Profit margin meets target? ◇
Yes: target met	No: target not met	▱ Display analysis
(END)		

Pseudocode

```
START
INPUT units, materialQty, materialRate, wastagePercent
INPUT labourCost, machineHours, machineRate, sellingPricePerUnit, targetMargin
IF any quantity/rate < 0 OR units <= 0 THEN
    OUTPUT "Invalid production data"; STOP
END IF
materialBase = materialQty * materialRate
wastageCost = materialBase * wastagePercent / 100
machineCost = machineHours * machineRate
totalCost = materialBase + wastageCost + labourCost + machineCost
unitCost = totalCost / units
revenue = units * sellingPricePerUnit
profit = revenue - totalCost
profitMargin = profit / revenue * 100 IF revenue > 0
IF profitMargin >= targetMargin THEN result = "TARGET MET"
ELSE result = "TARGET NOT MET"
OUTPUT costs, unitCost, revenue, profit, profitMargin, result
STOP
```

Sample Output

Metric	Value
Material cost	₹20,000
Wastage (5%)	₹1,000
Labour	₹8,000
Machine cost	₹3,000
Total production cost	₹32,000
Units / selling price	100 / ₹400
Revenue	₹40,000
Profit / margin	₹8,000 / 20%
Target (18%)	MET

Assumptions: Illustration assumes 100 units and a target profit margin of 18% of revenue.

9. Multi-Currency Travel Budget Analysis

Purpose: Convert expense categories using supplied exchange rates and fees; compare total with budget and find the largest category.

Inputs: Expense amounts by category and currency, exchange rates to base currency, conversion fee rate, total budget.

Algorithm

1. Read expenses, exchange rates, fee rate, and budget.
2. Validate non-negative expenses and positive exchange rates.
3. Convert each expense to base currency.
4. Apply conversion fee according to the policy and sum converted expenses plus fees.
5. Compare total with budget and identify the largest category by converted cost.

Flowchart

(START)	▱ Read expenses, rates, fee and budget	Validate amounts and rates
◇ Valid? ◇	▱ No: report error	Yes: convert each expense
Add conversion fees	Update total and largest category	◇ Total within budget? ◇
Yes: within budget	No: over budget	▱ Display totals and largest category
(END)		

Pseudocode

```
START
INPUT expensesByCategory, exchangeRates, feeRate, budget
totalSpend = 0
largestCategory = NONE
largestAmount = -1
FOR each expense
    IF expense.amount < 0 OR exchangeRate <= 0 THEN
        OUTPUT "Invalid expense or exchange rate"; STOP
    END IF
    converted = expense.amount * exchangeRate
    fee = converted * feeRate / 100
    categoryTotal = converted + fee
    totalSpend += categoryTotal
    IF categoryTotal > largestAmount THEN
        largestAmount = categoryTotal
        largestCategory = expense.category
    END IF
END FOR
difference = budget - totalSpend
IF difference >= 0 THEN status = "WITHIN BUDGET"
ELSE status = "OVER BUDGET"
OUTPUT totalSpend, difference, largestCategory, status
STOP
```

Sample Output

Category	Original	Rate to INR	Converted + fee
Hotel	USD 300	₹83/USD	₹25,647
Food	EUR 120	₹90/EUR	₹11,340
Transport	USD 80	₹83/USD	₹6,839.20
Total spend			₹43,826.20
Budget			₹45,000
Remaining			₹1,173.80

Category	Original	Rate to INR	Converted + fee
Largest category			Hotel

Assumptions: Illustrative rates and a 3% fee. Exchange rates are examples only; replace with the rates provided in the question or by the assignment.

10. Campus Event Financial Planning

Purpose: Calculate event costs, participant contributions, sponsorship gap, and minimum registrations for break-even.

Inputs: Venue, equipment, catering per person, transport per person, certificate per person, sponsorship, registration fee, expected participants.

Algorithm

1. Read fixed costs, per-participant costs, sponsorship, registration fee, and expected attendance.
2. Validate all costs and counts.
3. Calculate total fixed cost and variable cost per participant.
4. Calculate total event cost, available sponsorship, and funding shortfall.
5. Calculate the smallest whole number of registrations whose contributions plus sponsorship cover total cost.

Flowchart

(START)	▱ Read fixed/variable costs and funding	Validate inputs
◇ Valid? ◇	▱ No: report error	Yes: calculate fixed and per-person cost
Calculate total cost and available funds	Calculate shortfall	◇ Can each extra registration cover variable cost? ◇
No: check whether break-even is possible	Yes: calculate minimum break-even registrations	▱ Display budget and registration target
(END)		

Pseudocode

```
START
INPUT venueCost, equipmentCost
INPUT cateringPerPerson, transportPerPerson, certificatePerPerson
INPUT sponsorship, registrationFee, expectedParticipants
IF any cost < 0 OR counts < 0 THEN OUTPUT "Invalid input"; STOP
fixedCost = venueCost + equipmentCost
variablePerPerson = cateringPerPerson + transportPerPerson + certificatePerPerson
totalCost = fixedCost + variablePerPerson * expectedParticipants
availableFunds = sponsorship + registrationFee * expectedParticipants
shortfall = MAX(0, totalCost - availableFunds)
contributionPerPerson = registrationFee + sponsorship / MAX(1, expectedParticipants)
IF registrationFee <= variablePerPerson THEN
    IF sponsorship < fixedCost THEN
        OUTPUT "Break-even impossible at this registration fee"
    ELSE breakEvenCount = 0
    END IF
ELSE
    breakEvenCount = CEILING((fixedCost - sponsorship) /
                    (registrationFee - variablePerPerson))
    breakEvenCount = MAX(0, breakEvenCount)
END IF
OUTPUT totalCost, availableFunds, shortfall, breakEvenCount
STOP
```

Sample Output

Budget item	Amount
Venue + equipment (fixed)	₹20,000
Per participant (catering + transport + certificate)	₹250
Expected participants	100
Total event cost	₹45,000
Sponsorship	₹15,000

Budget item	Amount
Registration fee per person	₹400
Expected registration funds	₹40,000
Surplus	₹10,000
Break-even registrations (with sponsorship)	44

Assumptions: Example: fixed cost ₹20,000, variable cost ₹250/person, sponsorship ₹15,000, fee ₹400/person. Break-even count = ceiling((20,000−15,000)/(400−250)) = 34 registrations. The sample table's 44 figure would be inconsistent, so the algorithm should use 34; see corrected note below.